

Competitive Programming

Section 34: Probability and Expected Value

Competitive Programming Course

2026-07-10

Table of contents

1	What an expected value problem is asking	3
2	Exact rational answers modulo a prime	4
3	Pattern 1: linearity of expectation	5
3.1	Worked trace	5
4	Pattern 2: first-step equations	6
4.1	Worked trace: success probability $2/5$	6
5	Pattern 3: coupon collector as staged waiting	7
5.1	Worked trace: $n=3$	7
6	Precompute harmonic sums efficiently	8
7	Debugging first-step models	9
8	C++ and Python implementation notes	10
9	Formative exercises	11
10	Exercises	12
11	Local workflow and checklist	13

1 What an expected value problem is asking

Expectation is the average outcome over many independent repetitions of an experiment. It is not the most likely outcome. In contest problems it often lets us avoid listing exponentially many outcomes.

The three reliable entry points are:

Statement shape	First model to try
total reward is a sum of event rewards	indicator variables and linearity
repeat until a success	first-step equation / geometric wait
collect new kinds one by one	sum the waiting time of each stage

Write the random variable first. A formula without a random variable is hard to check and easy to apply to the wrong event.

2 Exact rational answers modulo a prime

The local problems ask for an exact rational number modulo $\text{MOD} = 1,000,000,007$, not a decimal approximation. Represent

a / b

by $a * b^{(\text{MOD}-2)} \bmod \text{MOD}$. This is valid because MOD is prime and every denominator used by the local contracts is nonzero modulo MOD .

For example, $1/3$ is the residue 333333336 , because

$$3 * 333333336 = 1,000,000,008 = 1 \pmod{\text{MOD}}.$$

Never use `double` and never use integer division. If a future problem allows a denominator divisible by MOD , this representation is not defined by a Fermat inverse; do not silently reuse this technique.

3 Pattern 1: linearity of expectation

For random variables X and Y ,

$$E[X + Y] = E[X] + E[Y].$$

Independence is **not** required for this equation. It is only needed for some probability products later in the course.

If bonus i has value $v[i]$ and happens with probability $p[i]/q[i]$, define

$$X_i = v[i] \text{ if bonus } i \text{ happens, otherwise } 0.$$

Then $E[X_i] = v[i] * p[i] / q[i]$, and the total reward is $\sum X_i$. Sum the individual expectations instead of enumerating all 2^n event subsets.

3.1 Worked trace

For bonuses $(p, q, v) = (1, 2, 10), (1, 4, 8), (3, 4, 12)$:

$$\begin{aligned} E[X_1] &= 10 * 1/2 = 5 \\ E[X_2] &= 8 * 1/4 = 2 \\ E[X_3] &= 12 * 3/4 = 9 \\ E[\text{total}] &= 5 + 2 + 9 = 16. \end{aligned}$$

The local problem describes independent events, but notice that the calculation above did not multiply event probabilities. It would remain valid if the bonus events were dependent and their individual probabilities stayed the same.

4 Pattern 2: first-step equations

Suppose one attempt succeeds with probability p . Let E be the expected number of attempts until the first success. We always spend one attempt. After a failure, probability $1-p$, we are back in the same state:

$$\begin{aligned} E &= 1 + (1-p)E \\ pE &= 1 \\ E &= 1/p. \end{aligned}$$

For a local input probability p/q , this is q/p modulo MOD .

4.1 Worked trace: success probability $2/5$

$$\begin{aligned} E &= 1 + (3/5)E \\ (2/5)E &= 1 \\ E &= 5/2. \end{aligned}$$

Modulo MOD , print `5 * inverse(2) = 500000006`. The contract for problem B has $1 \leq p \leq q < \text{MOD}$; success probability zero is intentionally excluded, because then no finite expectation exists. Treat that as a modeling warning, not as a hidden test case.

5 Pattern 3: coupon collector as staged waiting

There are n equally likely coupon types. When i distinct types are already collected, $n-i$ types would be new, so

$$P(\text{next draw is new}) = (n-i)/n.$$

The expected waiting time for the next new type is the reciprocal:

$$E[\text{stage } i] = n/(n-i).$$

The total time is the sum of the n stages, from zero types to all types:

$$\begin{aligned} & n/n + n/(n-1) + \dots + n/1 \\ &= n * (1 + 1/2 + \dots + 1/n). \end{aligned}$$

5.1 Worked trace: $n=3$

collected i	chance of new type	expected draws for this stage
0	3/3	1
1	2/3	3/2
2	1/3	3
total		11/2

Modulo MOD, 11/2 is 500000009. The stage expectation is valid even though the number of failed draws in different stages is not fixed; linearity adds the stage random variables.

6 Precompute harmonic sums efficiently

For up to 200,000 coupon queries with $n \leq 1,000,000$, compute

```
H[i] = H[i-1] + inverse(i),    H[0] = 0.  
answer(n) = n * H[n].
```

Calling binary exponentiation for every inverse costs $O(N \log \text{MOD})$ and can be acceptable in optimized C++, but a linear inverse table is the better reusable tool:

```
inv[1] = 1  
inv[i] = MOD - (MOD / i) * inv[MOD % i] mod MOD, for i >= 2.
```

This recurrence is valid for $1 \leq i < \text{MOD}$, which holds here. It gives $O(N)$ preprocessing and $O(1)$ per query.

7 Debugging first-step models

For an expectation state $E[\text{state}]$, write:

```
E[state] = immediate cost + sum P(next state) * E[next state].
```

If a transition returns to the same state, move that term to the left before coding. The geometric wait is the smallest example. If all transitions move to smaller states, compute the equations with DP instead.

8 C++ and Python implementation notes

In C++, keep every modular product in `long long`; reduce after each product. For Coupon Collector, allocate `vector<long long>` once through the maximum query and use fast buffered I/O. In Python, use `pow(x, MOD-2, MOD)` for a single inverse and keep harmonic/inverse tables in lists; read all queries first to know the required maximum. The linear inverse recurrence reduces Coupon Collector preprocessing from $O(N \log \text{MOD})$ to $O(N)$.

9 Formative exercises

1. A reward of 9 is gained with probability $2/3$, and a reward of 6 is gained with probability $1/2$. What is the expected total? Would your answer change if the two events were dependent?
2. An attempt succeeds with probability $1/4$. Write, then solve, the first-step equation for its expected number of attempts.
3. For three coupon types, what is the probability of a new type after exactly one distinct type has been collected? What is that stage's expectation?
4. Why is $p=0$ not a valid input for First Success, while $p=0$ is valid for an individual Expected Prize event?

Answers:

1. $9 \cdot (2/3) + 6 \cdot (1/2) = 9$. No: linearity does not require independence.
2. $E = 1 + (3/4)E$, so $E = 4$.
3. $2/3$, then $3/2$.
4. A zero-probability prize simply contributes expectation zero. Repeating an experiment that can never succeed has infinite expected stopping time.

10 Exercises

Complete the four formative exercises above before local Problems A–C. They move from linearity, to a self-loop equation, to staged expectation.

11 Local workflow and checklist

Run:

```
CP_TARGET=solution python3 sections/34_probability_expected_value/check.py
```

Before submission, check:

```
I defined a random variable or expectation state.  
I used linearity without adding an unnecessary independence assumption.  
I multiplied probabilities only where independence is stated.  
Every modular denominator is nonzero modulo MOD.  
The first-step equation includes the attempt made now.  
Coupon stages run from i=0 through n-1, not in the reverse direction.
```